Practical Session 19th of November

assignment sociolinguistics

Solution Part 1

Sociolinguistics

Goal

Overarching Goal:

→ create a new and enriched version of the two reddit corpora, gen-z and ask-old-people

Methods:

pre-processing:

- tokenization
- linguistic features
- additional features (gen-z expression occurrences)

- creating / processing corpus objects in convokit

Solution

Methods:

pre-processing:

- tokenization

```
from convokit.text_processing import TextParser

# create a tokenizer and apply it to the corpora
tokenizer = TextParser(mode="tokenize")
corpus_old_tokenized = tokenizer.transform(corpus_old)
corpus_young_tokenized = tokenizer.transform(corpus_young)
```

→ this takes long for larger dataset

→ output is stored in the 'parsed' field, add another field 'num_tokens'

```
for utt in corpus_old_tokenized.iter_utterances():
    sentences = utt.meta['parsed']
    num_tokens = sum(len(sent['toks']) for sent in sentences)
    utt.meta['num_tokens'] = num_tokens
for utt in corpus_young_tokenized.iter_utterances():
    sentences = utt.meta['parsed']
    num_tokens = sum(len(sent['toks']) for sent in sentences)
    utt.meta['num_tokens'] = num_tokens
```

Solution

Methods:

- processing / filtering

```
utterances_genz = corpus_young_tokenized.get_utterances_dataframe()
filtered_utterances_genz = utterances_genz[
    (utterances_genz['meta.num_tokens'] > 10) & (utterances_genz['meta.num_tokens'] < 350)]
```

→ filter based on newly created field

→a new corpus object can be created with a list of utterance objects

```
utterance_objects_genz = []
for utt_id in filtered_utterances_genz.index:
    utt = corpus_young_tokenized.get_utterance(utt_id)
    utterance_objects_genz.append(utt)
corpus_genz_filtered = Corpus(utterances=utterance_objects_genz)
```

Solution

Methods:

- linguistic features

```
import spacy
nlp = spacy.load("en_core_web_sm")
# process utterances with spacy pipe
processed_utterances_genz = list(nlp.pipe(utterance_samples_genz))
```

→ use spacy to process list of utterances result is a list of spacy documents

→use lftk Extractor object to extract linguistic features. The Extractor required spacy documents as input. Feature extraction is done under the hood (here the library uses additional information from spacy, e.g. pos-tags to calculate feature values)

```
import lftk
LFTK = lftk.Extractor(docs=processed_utterances_genz)
print("LFTK initialized for Gen Z utterances.")
features_genz_sample = LFTK.extract(features=features_to_extract)
```

→ for a larger dataset we want to do that on a server

Solution

Methods:

- linguistic features - normalization

```
for utt in corpus_genz_filtered.iter_utterances():
    utt_id = utt.id
    features_utterance = features_genz.loc[utt_id]

    for feature in features_genz.columns:
        value = features_utterance[feature]
        # normalize if needed
        if feature.startswith("n_") and not feature.startswith("n_u"):
            value = value / utt.meta['num_tokens']
        elif feature.startswith("n_u"):
            corresponding_n_feature = feature.replace("n_u", "n_")
            n_value = features_utterance[corresponding_n_feature]
            if n_value > 0:
                value = value / n_value
            else:
                value = 0.0
        utt.meta[feature] = value
```

→ iterate over utterances. locate the corresponding feature values for each feature based on the given dataframe index and utterance id. normalize the value

Solution

Methods:

- additional features

```
# load genz wordlist
with open("/Users/falkne/PycharmProjects/nlpforSocialInteractions/additional_data/gen_z_wordlist.txt", "r") as f:
    genz_wordlist = f.read().splitlines()
print("First 10 words in the genz wordlist:", genz_wordlist[:10])
```

```
# function to count genz expressions in an utterance
def count_gen_z_words(utterance):
    tokens = utterance.meta['parsed'][0]['toks']
    tokens = [t["tok"].lower() for t in tokens]
    count_unigrams = sum(1 for t in tokens if t in genz_wordlist)
    full_string = " ".join(tokens)
    count_multigrams = 0
    for gen_z_word in genz_wordlist:
        if " " in gen_z_word:
            count_multigrams += full_string.count(gen_z_word)
    return count_unigrams + count_multigrams
```

→simplified method to count the occurrences of uni- and n-grams. Many possibilities here…

Expected Output

```
Sample utterance meta information after adding features: ConvoKitMeta({'score': 1, 'top_level_comment': 'e29tjhv',
'retrieved_on': 1537936910, 'gilded': 0, 'gildings': {'gid_1': 0, 'gid_2': 0, 'gid_3': 0}, 'subreddit': 'GenZ',
'stickied': False, 'permalink': '/r/GenZ/comments/8ybems/what_is_your_favourite_movie/e4ueb27/', 'author_flair_text':
'1998', 'parsed': [{'toks': [{'tok': 'That'}, {'tok': 'movie'}, {'tok': 'will'}, {'tok': 'break'}, {'tok': 'you'},
{'tok': '.'}]}, {'toks': [{'tok': 'Guaranteed'}, {'tok': '.'}]}, {'toks': [{'tok': 'One'}, {'tok': 'of'}, {'tok':
'the'}, {'tok': 'most'}, {'tok': 'haunting'}, {'tok': 'films'}, {'tok': 'ever'}, {'tok': 'made'}, {'tok': 'if'},
{'tok': 'you'}, {'tok': 'ask'}, {'tok': 'me'}, {'tok': '.'}]}], 'num_tokens': 21, 'num_genz_words': 0, 'n_adj':
np.float64(0.047619047619047616), 'n_adp': np.float64(0.047619047619047616), 'n_adv': np.float64(0.09523809523809523),
'n_aux': np.float64(0.047619047619047616), 'n_cconj': np.float64(0.0), 'n_det': np.float64(0.09523809523809523),
'n_intj': np.float64(0.0), 'n_noun': np.float64(0.09523809523809523), 'n_num': np.float64(0.047619047619047616),
'n_part': np.float64(0.0), 'n_pron': np.float64(0.14285714285714285), 'n_propn': np.float64(0.0), 'n_punct': np.float64
(0.14285714285714285), 'n_sconj': np.float64(0.047619047619047616), 'n_sym': np.float64(0.0), 'n_verb': np.float64(0
.19047619047619047), 'n_space': np.float64(0.0), 'n_uadj': np.float64(1.0), 'n_uadp': np.float64(1.0), 'n_uadv':
np.float64(0.5), 'n_uaux': np.float64(1.0), 'n_ucconj': 0.0, 'n_udet': np.float64(0.5), 'n_uintj': 0.0, 'n_unoun':
np.float64(0.5), 'n_unum': np.float64(1.0), 'n_upart': 0.0, 'n_upron': np.float64(0.6666666666666666), 'n_upropn': 0.0,
 'n_upunct': np.float64(0.3333333333333333), 'n_usconj': np.float64(1.0), 'n_usym': 0.0, 'n_uverb': np.float64(0.25),
'n_uspace': 0.0, 'a_word_ps': np.float64(7.0), 'a_bry_ps': np.float64(4.667), 'corr_ttr': np.float64(0.463)})
```

Sociolinguistic Analysis

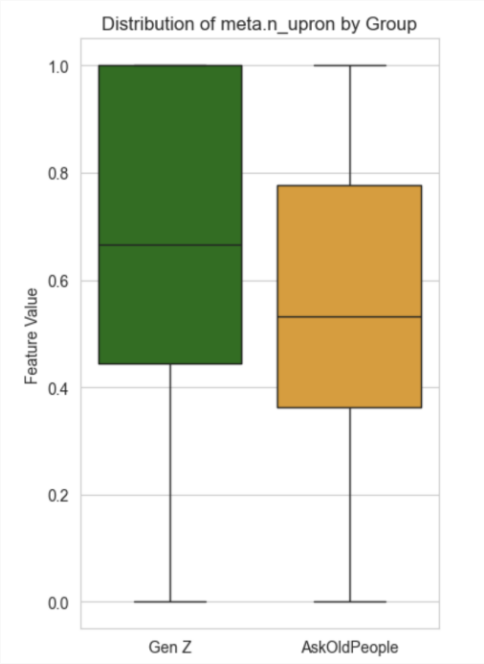
Task 1:
- compare the distributions / values for each feature between the two corpora, find features that point to interesting differences
- discuss the results in a smaller group

https://padlet.com/kbm54vknmk/results-assignment-sociolinguistics-gpai0io3rwyaub7z

Which features show the biggest differences between the two age groups?

What do these differences tell you about the language use of younger versus older speakers?

How to compare distributions?



Boxplot Visualization

```
import seaborn as sns

sns.boxplot(
    data=data,
    x="Subreddit",
    y="Feature Value",
    palette=["green", "orange"],
)
```

How to compare distributions?

Boxplot Visualization for a summary

bottom whisker: what are the minimum (non-outlier) values?

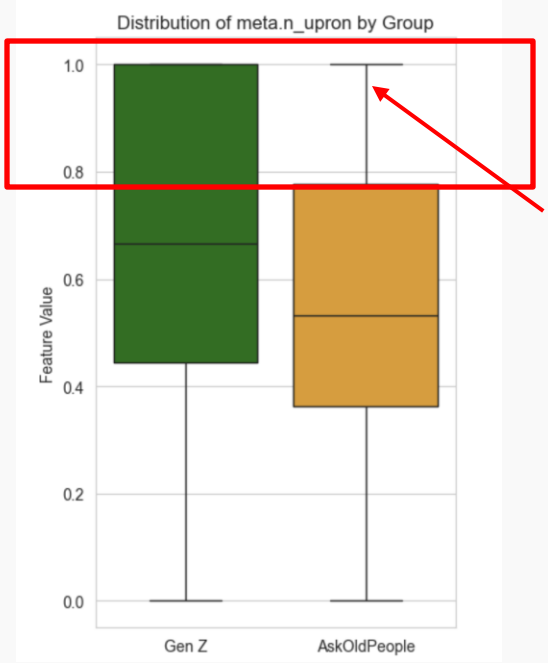
How to compare distributions?

Boxplot Visualization for a summary

bottom whisker: what are the minimum (non-outlier) values?

→ in both corpora some posts use zero unique pronouns

How to compare distributions?



Boxplot Visualization for a summary

upper whisker: what are the maximum (non-outlier) values?

→ in both corpora there are posts that have 100% unique pronouns (that means all pronouns in a posts are different or there is
just one pronoun that occurs)

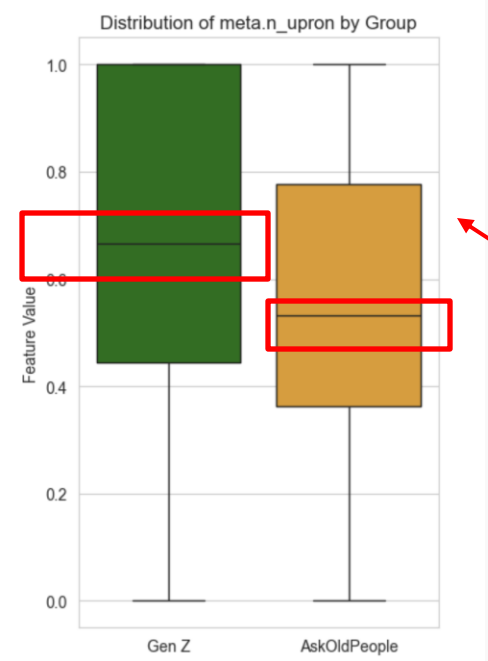
How to compare distributions?

Boxplot Visualization for a summary

the **box**: interquartile range: 50% of the datapoints have values inside the box

→they look different for our two corpora: larger box = data is more spread (more variability in Gen Z). More Data Points in Gen Z have a higher proportion of unique pronouns

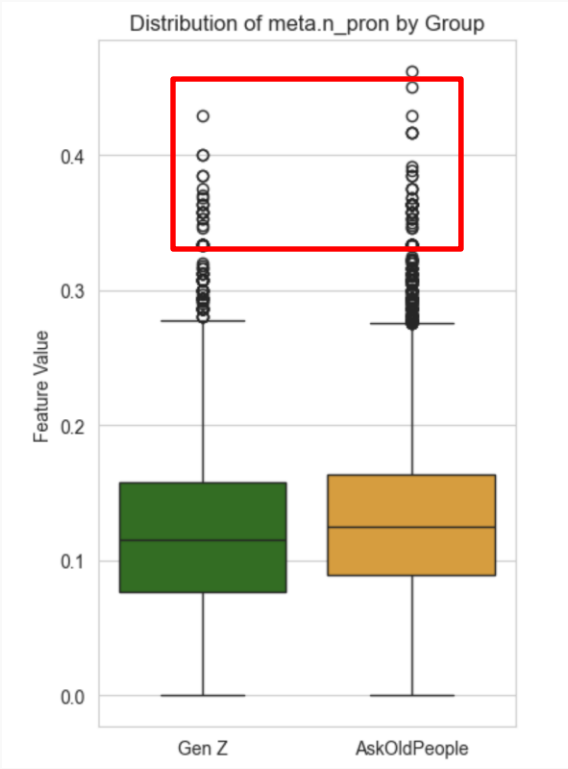
How to compare distributions?



Boxplot Visualization for a summary

the **median** (Q2): 50th percentile
→significantly higher for Gen-Z so the proportion of unique pronouns is higher in the Gen-Z corpus

How to compare distributions?

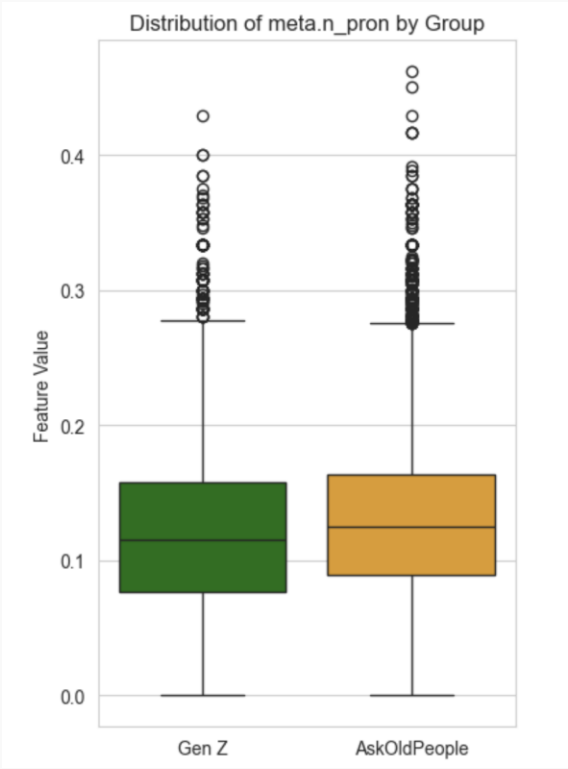


Boxplot Visualization for a summary

compare unique pronoun use to general pronoun use

outliers: some people use a very high amount of pronouns in their utterances

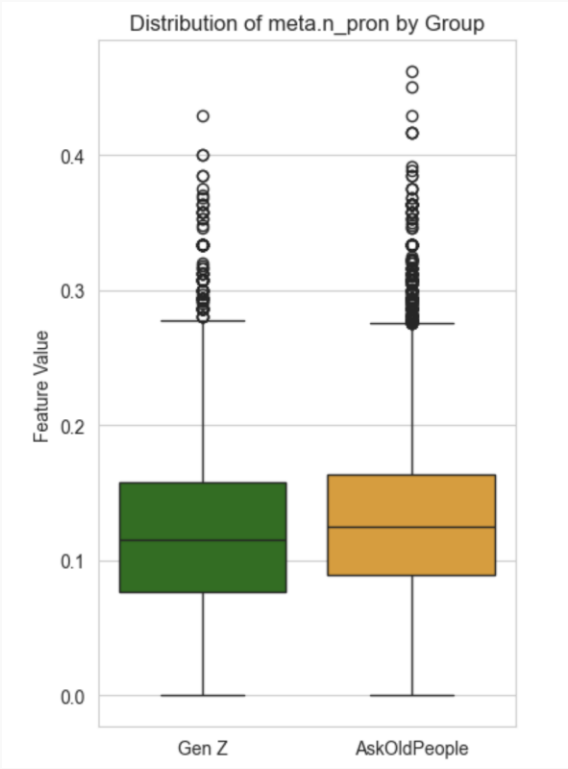
How to compare distributions?



Boxplot Visualization for a summary

→ we can see that the use of pronouns is very similar (slightly higher amount of pronouns in AskOldPeople) → more variation in pronoun use in the Gen Z forum compared to askOldPeople

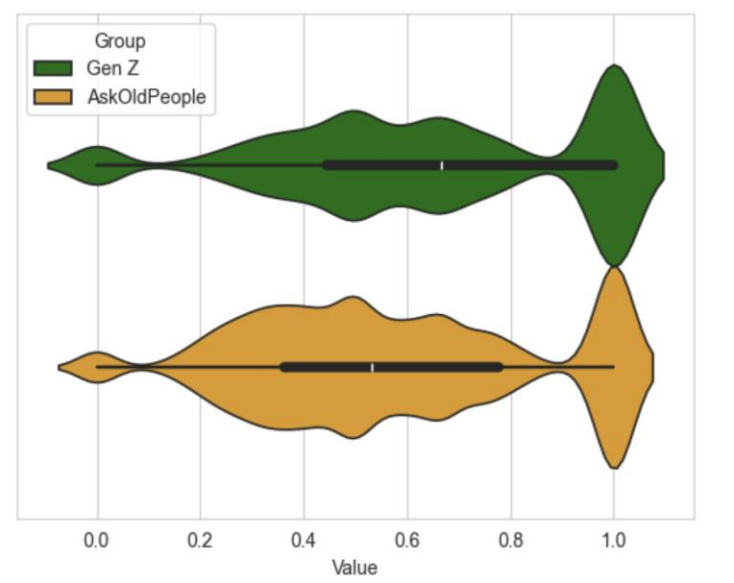
How to compare distributions?



Boxplot Visualization for a summary

→drawback boxplot:
- we don't know how the underlying distribution looks like
- does not take sample size into account (they tell you the range within which values occur but not how many occur in each region

How to compare distributions?



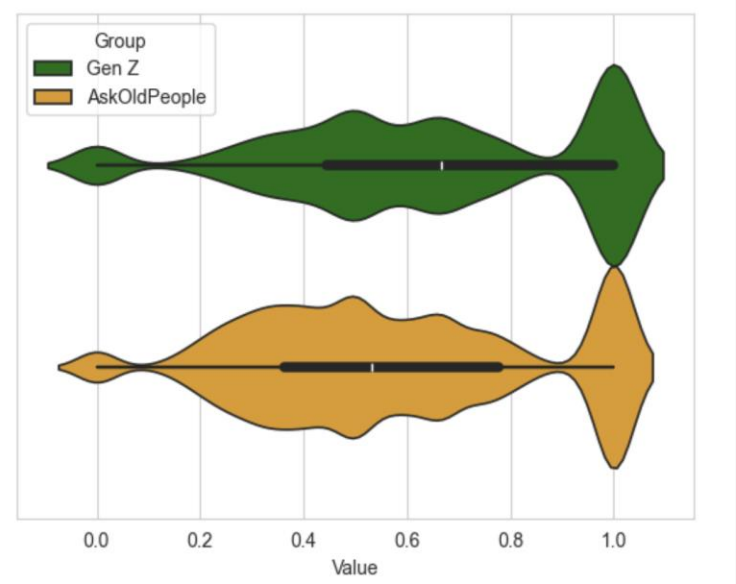
alternative: violin plot

```
sns.violinplot(data=data, x='feature value',
hue=subreddit', hue_order=['Gen Z',
'AskOldPeople'], palette=['green', 'orange'],
inner='box', linewidth=1.2)
```

→ contains the information of the box plot but also shows the distribution

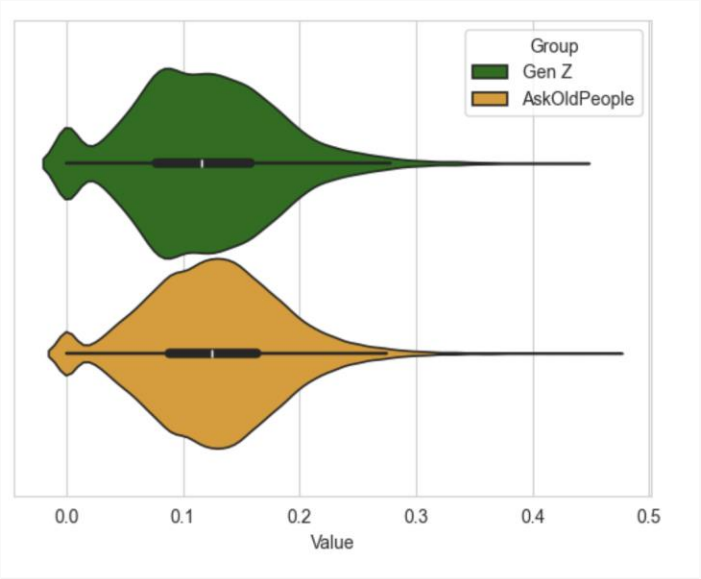
proportion unique pronouns

How to compare distributions?



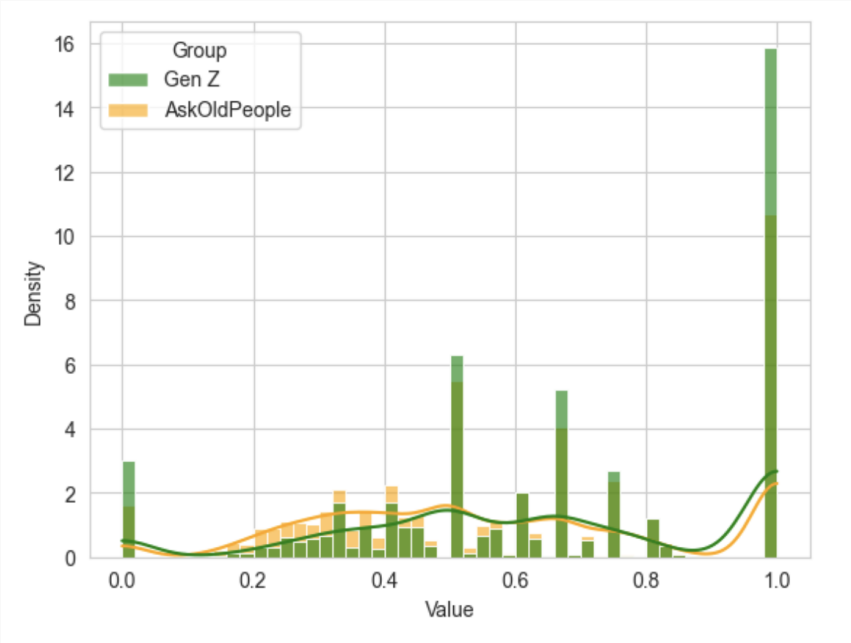
proportion unique pronouns

alternative: violin plot



proportion pronouns within an utterance

How to compare distributions?

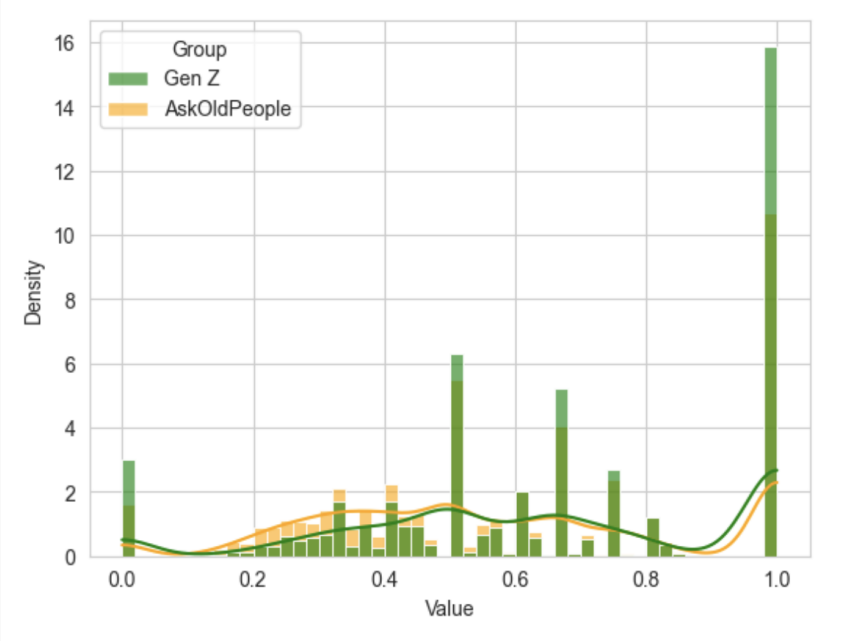



```
sns.histplot(x=´feature Value', hue=´subreddit',
data=data, bins=50, kde=True, stat="density",
palette=["green", "orange"], alpha=0.6,
common_norm=False)
```

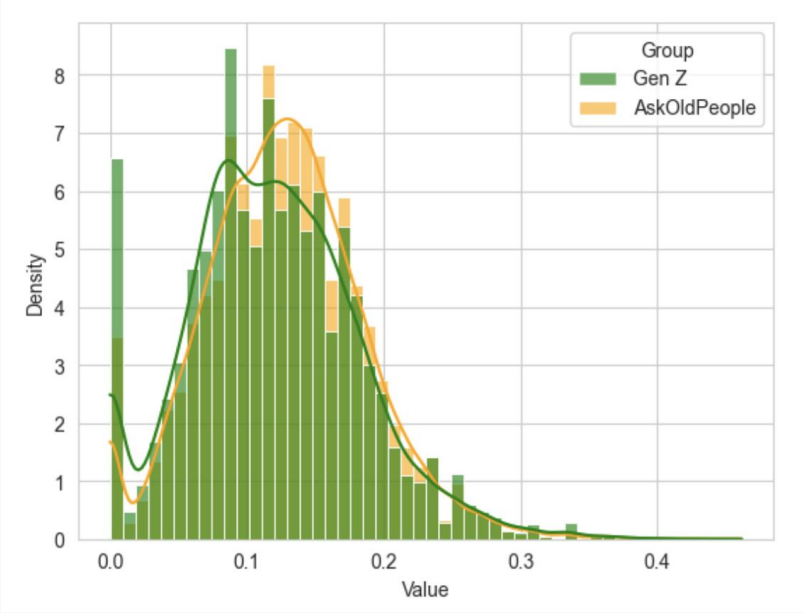
proportion unique pronouns

proportion pronouns within an utterance

How to compare distributions?



proportion unique pronouns



proportion pronouns within an utterance

How to compare distributions?

are values in one group significantly higher than in the other?

Mann–Whitney U test (no assumptions about normality, different sample sizes do not matter)

```
from scipy.stats import mannwhitneyu
x = dataframe_genz['meta.n_pron'].tolist()
y = dataframe_old['meta.n_pron'].tolist()
U, p = mannwhitneyu(x, y, alternative='two-sided')
# and effect size
rank_biserial_corr = 1 - (2 * U / (len(x) * len(y)))
# print .3f
print(f"Mann-Whitney U test: U={U}, p={p:.3f}")
print(f"Rank-biserial correlation: {rank_biserial_corr:.3f}")
```

p-val: whether the difference is significant
rank_biserial_corr: effect size

- ≈ 0.10 → Small effect
- ≈ 0.30 → Medium effect
- ≥ 0.50 → Large effect

How to compare distributions?

are values in one group significantly higher than in the other?

comparing unique pronouns: are values in gen-z higher?

p-val: whether the difference is significant
rank_biserial_corr: effect size

```
Mann-Whitney U test: U=134507163.0, p=0.000
Rank-biserial correlation: -0.154
```

- ≈ 0.10 → Small effect
- ≈ 0.30 → Medium effect
- ≥ 0.50 → Large effect

How to find distinct word use

```
from sklearn.feature_extraction.text import CountVectorizer
from convokit import FightingWords
cv = CountVectorizer(ngram_range=(1,1), stop_words='english')
# import Counter
fw = FightingWords(ngram_range=(1,1),cv=cv)
```

this is optional, if not specified uses default values for the CountVectorizer

How to find distinct word use

```
from sklearn.feature_extraction.text import CountVectorizer
from convokit import FightingWords
cv = CountVectorizer(ngram_range=(1,1), stop_words='english')
# import Counter
fw = FightingWords(ngram_range=(1,1),cv=cv)
```

```
fw.fit(combined_corpus, class1_func=lambda utt: utt.meta['subreddit'] == 'genZ',
                        class2_func=lambda utt: utt.meta['subreddit'] == "askoldpeople")
df = fw.summarize(combined_corpus, plot=True, class1_name='r/genz', class2_name='r/askoldpeople')
```